

VYAPAR SATHI DATABASE AND INTERVIEW GUIDE

1. Project database overview

Which databases are used in the project?

? MySQL is the main relational database.

? ChromaDB is the vector database used for GST knowledge retrieval.

? The filesystem also stores uploaded files in the 'uploads/' folder, but that is not a database.

Where is MySQL configured?

? 'app/core/database.py' creates the SQLAlchemy engine and session.

? '.env' contains 'DATABASE_URL=mysql+pymysql://...'.

Where is ChromaDB used?

? 'app/services/rag_service.py' creates

'chromadb.PersistentClient(path="modules/module_1_rag/chroma_db")'.

? PDF chunks are added to the 'knowledge_base' collection.

How the full project works step by step

1. The React frontend calls FastAPI endpoints.
2. Auth, transactions, chat, and uploads go to the backend.
3. Structured records are stored in MySQL.
4. GST PDFs are chunked, embedded, and stored in ChromaDB.
5. Business questions use MySQL.
6. GST knowledge questions use ChromaDB.
7. The assistant combines retrieval logic with final answer formatting.

2. What MySQL stores

? Users

? User profiles

? Conversations

? Messages

? Transactions

? Uploaded file metadata

? System logs

Important tables

? 'users'

? 'user_profiles'

? 'conversations'

? 'messages'

? 'transactions'

? 'user_documents'

? 'system_logs'

Why MySQL is used

- ? The app has structured business data.
- ? The app needs relations between users, chats, and transactions.
- ? The app needs totals, counts, GST sums, profit, and date filters.
- ? MySQL is a strong fit for transactional and reporting workloads.

Why not only MySQL

- ? MySQL is not naturally built for semantic similarity retrieval over long GST text.
- ? It can store embeddings, but it is not the best tool to search them by meaning in this project.

3. What ChromaDB stores

- ? GST document chunks
- ? Embeddings for those chunks
- ? Vector index data for similarity search

Why ChromaDB is used

- ? It supports semantic retrieval.
- ? It is easier to use than a low-level vector library for this project.
- ? It provides persistent local storage.
- ? It fits RAG workflows well.

Why not only ChromaDB

- ? ChromaDB is not a replacement for users, transactions, chat history, and financial reporting.
- ? It is good for vector search, not for general business CRUD and relational queries.

4. Why not MongoDB?

- ? This project behaves like a finance and ledger system.
- ? The main data is structured and relational.
- ? The app needs exact schema, filtering, aggregation, and consistent reporting.
- ? MySQL is a better fit than MongoDB for this workload.
- ? For GST semantic retrieval, ChromaDB is more suitable than MongoDB.

5. How to decide chunk size in ChromaDB

- ? Chunk size is chosen based on how much context each chunk should keep.
- ? If chunks are too small, meaning gets broken.
- ? If chunks are too large, retrieval becomes noisy and less precise.
- ? For GST and policy PDFs, medium-size paragraph-level chunks are usually better.

Good rule for this project

- ? '700-900' characters: more focused retrieval
- ? '1000-1200' characters: better context preservation
- ? Current code uses 'chunk_size=1000', which is a good default for GST documents.

6. What is chunk overlap?

? Chunk overlap means repeating some text from the end of one chunk at the start of the next chunk.

? Example: if chunk size is '1000' and overlap is '150', then the next chunk reuses the last '150' characters of the previous chunk.

Why overlap is useful

? It avoids losing meaning at chunk boundaries.

? It helps when a rule or sentence continues across chunks.

Why overlap should not be too large

? Too much duplication

? More storage

? More repeated retrieval results

7. Why not FAISS or a normal database for embeddings?

Why not FAISS in this project?

? FAISS is mainly a vector search library.

? ChromaDB is more application-ready for a local RAG setup.

? With FAISS, you usually manage storage, IDs, metadata, and persistence more manually.

? ChromaDB reduces glue code.

Why not a normal database for embeddings?

? A normal relational database can store vectors as text or blobs.

? But vector similarity search is not its main strength.

? This project needs meaning-based retrieval, not just storage.

8. How ChromaDB works internally

1. PDF text is loaded.

2. Text is split into chunks.

3. Each chunk is converted into an embedding vector.

4. ChromaDB stores chunk text, IDs, and embeddings.

5. ChromaDB maintains a vector index for fast similarity retrieval.

6. At query time, the user query is also embedded.

7. ChromaDB finds the nearest chunk vectors and returns the matching chunks.

8. The app builds the final short answer from those chunks.

Important idea

? ChromaDB does not directly generate the answer.

? It retrieves the most relevant chunks.

? Your application logic formats the final answer.

9. How vector index building happens

1. Chunk text is prepared.

2. An embedding model converts each chunk into a vector.

3. The vectors are stored in the collection.
4. ChromaDB organizes them into a search-friendly structure.
5. This structure helps nearest-neighbor retrieval happen faster than comparing every vector one by one.

Simple meaning

? Storing vectors is not the same as indexing vectors.

? Storage saves them.

? Indexing organizes them for fast similarity search.

10. Important interview questions with brief answers

Q1. Which databases are used in your project?

My project uses MySQL for structured business data and ChromaDB for GST document embeddings and semantic retrieval.

Q2. Why did you use two databases instead of one?

Because the project has two different workloads: structured business operations and unstructured semantic knowledge retrieval.

Q3. Why did you choose MySQL?

Because users, transactions, chat history, and reports are structured and relational, and MySQL handles aggregates and consistency well.

Q4. Why did you choose ChromaDB?

Because the GST assistant needs vector search over document chunks, and ChromaDB is a good local persistent RAG database.

Q5. Why not MongoDB?

Because the main application behaves more like a ledger and reporting system, which is a better fit for MySQL.

Q6. Why not only MySQL?

Because semantic retrieval over GST text is better handled by a vector database.

Q7. Why not only ChromaDB?

Because it is not designed to replace relational storage for users, transactions, and reports.

Q8. Why not FAISS?

FAISS is powerful but lower-level. ChromaDB is easier to integrate and persist for this project.

Q9. What data is stored in MySQL?

Users, profiles, chat sessions, messages, transactions, document metadata, and system logs.

Q10. What data is stored in ChromaDB?

GST knowledge chunks, embeddings, and vector index data.

Q11. What is a vector database?

A vector database stores embeddings and supports similarity search by meaning.

Q12. What is an embedding?

An embedding is a numeric vector representation of text meaning.

Q13. What is chunking?

Chunking is splitting a large document into smaller pieces before embedding and retrieval.

Q14. What is chunk overlap?

It is repeated text between neighboring chunks to preserve context.

Q15. How do you decide chunk size?

By balancing context preservation and retrieval precision. GST documents usually need medium-sized chunks.

Q16. What happens when a user asks 'What is my total sales'?

The system routes the query to MySQL and calculates the result from the 'transactions' table.

Q17. What happens when a user asks a GST rule question?

The system queries ChromaDB, retrieves relevant GST chunks, and then formats an answer.

Q18. How does the assistant decide between MySQL and ChromaDB?

It classifies the query based on whether it is a business-data question or a knowledge-base question.

Q19. What is the role of SQLAlchemy in your project?

SQLAlchemy connects the FastAPI backend to MySQL and maps Python models to database tables.

Q20. What are primary keys and foreign keys in your schema?

Primary keys uniquely identify records. Foreign keys connect related records like users to transactions or conversations to messages.

Q21. Why is MySQL good for financial reports?

Because it supports exact filters, sums, counts, date queries, and structured aggregation reliably.

Q22. What is semantic search?

Semantic search finds relevant text by meaning, not only by exact keyword match.

Q23. What is the difference between exact search and semantic search?

Exact search matches words. Semantic search matches meaning.

Q24. Does ChromaDB store final answers?

No. It stores chunks and embeddings. The application generates final answers.

Q25. Can one database be enough for this project?

Only if both workloads were similar. In this project, relational finance data and semantic GST retrieval need different storage strategies.

11. Fast viva answers for your project

Explain the database architecture of your project.

My project uses MySQL as the main operational database and ChromaDB as the vector database. MySQL stores structured business records like users, chats, and transactions, while ChromaDB stores GST document embeddings for semantic retrieval.

Why is MySQL used here?

Because the project needs structured storage, relationships, and reporting queries such as total sales, expenses, GST, and profit.

Why is ChromaDB used here?

Because GST question-answering needs vector search over document chunks, which is different from normal relational queries.

Why not MongoDB?

Because the project's core data is structured and reporting-heavy, so MySQL is a better match.

Why not one database only?

Because business records and semantic document retrieval are two different technical problems.

12. Best points to remember before interview

? MySQL is the main database.

? ChromaDB is the vector database.

? MySQL handles structured business data.

? ChromaDB handles GST semantic retrieval.

? MongoDB was not chosen because the app is relational and report-heavy.

? FAISS was not chosen because ChromaDB is easier to use as a persistent vector database in this project.

? Chunk size controls context length.

? Chunk overlap preserves context across chunk boundaries.

? Embeddings represent meaning numerically.

? Vector indexing speeds up nearest-neighbor retrieval.